NLP Project Report

Field	Detail
Group Number	Group 20
Group Members	James Luigi C. Capuz Julian Ivan S. De Guzman
Dataset	Rotten Tomatoes Movie Review Dataset (Binary Sentiment)
Source	Hugging Face – cornell-movie-review-data/rotten_tomatoes (Pang & Lee, 2005)
Train Subset	1,000 reviews (sampled from training split)
Validation Subset	200 reviews
Test Subset	200 reviews
GAN Subset	Training subset re-used with a custom 1,500-word vocabulary
Framework	PyTorch + Hugging Face Transformers
Device	CPU

1. Introduction and Objectives

This report documents the implementation, training, and evaluation of three distinct NLP model architectures applied to a common textual dataset, run end-to-end with results captured directly from the project notebook. The primary goal is to compare the operational mechanisms, empirical performance, and structural behavior of three fundamental paradigms in natural language processing:

- **BERT (Bidirectional Encoder Representations from Transformers):** a bidirectional representation encoder fine-tuned for discriminative text classification (sentiment analysis).
- **GPT-2 (Generative Pre-trained Transformer 2):** an autoregressive decoder, fine-tuned as distilgpt2, used for causal language modeling and text continuation.
- **Text-GAN (Generative Adversarial Network):** a custom sequence-based network using an LSTM generator and a 1D-CNN discriminator, connected through a continuous soft-embedding approximation that bridges the discrete-gradient barrier.

Each model was trained and evaluated on subsets of the Rotten Tomatoes movie review corpus, with standardized statistical metrics applied to enable a fair comparison across the three paradigms.

2. Dataset Description

The dataset used is the Rotten Tomatoes movie review dataset, sourced from the Hugging Face repository cornell-movie-review-data/rotten_tomatoes. Each review is labeled as either negative (class 0) or positive (class 1). To keep training times practical on CPU, a sampled subset was extracted from the original train, validation, and test splits.

Split	File	Rows	Purpose
Train	train.parquet (sampled)	1,000	BERT + GPT-2 fine-tuning; Text-GAN vocabulary and generator training
Validation	validation.parquet (sampled)	200	Validation loss / perplexity tracking; Text-GAN discriminator validation
Test	test.parquet (sampled)	200	Evaluation for all models

The dataset was loaded directly from local parquet files, and label distribution was inspected prior to sampling to keep the train, validation, and test subsets representative of the original balance between positive and negative reviews.

3. Variant 1: BERT (Text Classification)

3.1 Architecture and Configuration

The BERT variant uses the pre-trained bert-base-uncased model with a linear sequence classification head for binary sentiment analysis. BERT is a bidirectional encoder that reads sequences in both directions simultaneously using self-attention, so every token builds a contextual representation informed by both its left and right neighbors. Tokenization uses WordPiece (BertTokenizerFast) with a 30,522-token vocabulary, which splits out-of-vocabulary

words into morphological subwords (e.g., “unbelievable” → “un”, “##believable”), minimizing unknown-token issues.

Parameter	Value
Pre-trained model	bert-base-uncased
Tokenizer	WordPiece (BertTokenizerFast, 30,522-token vocabulary)
Max sequence length	128 tokens
Training epochs	1 (single fine-tuning run, CPU)
Train / Test samples	1,000 train / 200 test

3.2 Evaluation Results

After one fine-tuning epoch on CPU (372.7 seconds), the BERT model achieved the following results on the held-out test subset:

Metric	Value
Precision	0.8621
Recall	0.7075
F1-Score	0.7772
Training Time	372.7 s (1 epoch, CPU)

The model shows high classification precision from fine-tuning a pre-trained bidirectional encoder: few of its positive predictions are wrong. The comparatively lower recall indicates that a noticeable share of positive reviews were missed, which is consistent with only a single epoch of fine-tuning on a relatively small 1,000-review training subset.

4. Variant 2: GPT-2 (Causal Language Modeling)

4.1 Architecture and Configuration

The GPT-2 variant uses DistilGPT-2, a lightweight, 82-million-parameter distilled version of GPT-2, fine-tuned for causal language modeling on the Rotten Tomatoes review corpus. Unlike BERT, GPT-2 is a unidirectional decoder that models language causally, predicting the next token from left to right; self-attention is masked so that a token at position t can only attend to tokens at positions before it. Tokenization uses byte-level Byte-Pair Encoding (BPE) with a 50,257-token vocabulary, which eliminates out-of-vocabulary tokens entirely and handles arbitrary text structure natively.

Parameter	Value
Pre-trained model	distilgpt2
Tokenizer	Byte-level BPE (GPT2TokenizerFast, 50,257-token vocabulary)

Parameter	Value
Max sequence length	64 tokens
Training epochs	1 (single fine-tuning run, CPU)
Evaluation prompts	First 3 words of 50 held-out test sentences

4.2 Evaluation Results

Total training time was 184.6 seconds for one epoch. The generative quality metrics, computed by comparing generated continuations against held-out reference text, are as follows:

Metric	Value
Perplexity	81.16
Average BLEU	0.0653
Average ROUGE-L	0.2243
Training Time	184.6 s (1 epoch, CPU)

The model produced fluent, grammatically coherent continuations despite only limited fine-tuning, since the pre-training of distilgpt2 already provides strong linguistic priors that a from-scratch model could not match in a single epoch. Sample continuations, generated from the first three words of held-out test reviews, stayed thematically on-topic with romantic relationships, cinematic imagery, and movie-review language, even though the underlying logic was sometimes circular or repetitive:

Prompt	Generated Output
“the main story”	“...of this story is the only time you’ve thought of a romantic relationship where romantic love is the only means of ending up...”
“some motion pictures”	“...as well as some cinematic imagery... that was so clever, they ended up completely blank... and nothing at all really”
“there is a”	“...very strong and funny movie that is not just funny but a lot funny... well, well... you could argue, but”

5. Variant 3: Text-GAN (Adversarial Text Generation)

5.1 Architecture and Configuration

The Text-GAN variant uses a custom lightweight architecture consisting of an LSTM-based generator and a 1D-CNN discriminator. Standard GAN backpropagation fails on text because token selection via argmax or categorical sampling is discrete and non-differentiable. To resolve this, the generator outputs a continuous softmax probability vector at each step, which is multiplied directly by the shared embedding matrix to produce a sequence of soft embeddings. The discriminator classifies these soft embeddings against real one-hot embeddings, which preserves differentiability and allows end-to-end backpropagation.

Parameter	Value
Vocabulary size	1,504 tokens (top 1,500 words + <pad>, <unk>, <start>, <end>)
Embedding dimension	64
Generator hidden dimension	128
Generator type	LSTM, driven by latent noise z (latent dimension 100)
Discriminator type	1D-CNN over token embeddings
Max sequence length	15 tokens
Training epochs	15
Learning rate (G and D)	1e-3 (Adam, betas = 0.5, 0.999)
Loss function	BCELoss

5.2 Evaluation Results

Total training time was approximately 17.1 seconds for 15 epochs — roughly 20× faster per run than BERT, since the Text-GAN uses small custom embeddings rather than a pre-trained transformer. The discriminator was evaluated on the validation split, and the generated sequences were scored against reference text:

Metric	Value
Discriminator Accuracy	1.0000
Average BLEU	0.0000
Average ROUGE-L	0.0028
Training Time	17.1 s (15 epochs, CPU)

The Text-GAN was extremely fast to train but collapsed to repeating a narrow set of tokens, a textbook case of mode collapse. A discriminator accuracy of 1.0 signals that it learned to trivially separate soft (fuzzy, distributed) generated embeddings from real, sharp one-hot embeddings, rather than learning anything about grammar or semantics. Every generated sample converged to the same short, repetitive sequence:

Sample	Generated Output
Sample 1	“patient hair hair score pictures pictures pictures pictures pictures...”
Sample 2	“patient hair hair score pictures pictures pictures pictures pictures...”
Sample 3	“patient hair hair score pictures pictures pictures pictures pictures...”

6. Performance Evaluation Matrix

The table below compiles the metrics actually collected when the notebook was executed end-to-end: a single fine-tuning epoch on CPU for BERT and GPT-2, and a 15-epoch run for the Text-GAN.

Model Variant	Precision / Recall / F1	BLEU / ROUGE-L / Perplexity	Training Time	Key Observations
BERT (Classification)	0.8621 / 0.7075 / 0.7772	N/A (classification)	372.7 s (1 epoch, CPU)	High precision from fine-tuning a pre-trained bidirectional encoder; lower recall suggests some positive reviews were missed after only one epoch.
GPT-2 (Generation)	N/A (generative)	0.0653 / 0.2243 / 81.16	184.6 s (1 epoch, CPU)	Fluent, grammatically coherent continuations despite limited fine-tuning; pre-training gives distilgpt2 strong linguistic priors.
Text-GAN (Adversarial)	Disc. Accuracy: 1.0000	0.0000 / 0.0028 / N/A	17.1 s (15 epochs, CPU)	Extremely fast to train (~20× faster than BERT per run) but collapsed to a narrow, repetitive token sequence (mode collapse).

The center comparison confirms the large gap in computational cost between the architectures: the Text-GAN trains far faster per run than BERT or GPT-2, since it uses small custom embeddings rather than a pre-trained transformer. At the same time, GPT-2 substantially outperforms the Text-GAN on both BLEU and ROUGE-L, confirming that fluent generation quality comes at the cost of training time and model size.

7. Analytical Discussion

7.1 How Tokenization Differences Affected the Three Models

Model	Tokenizer	Strategy	Impact
BERT	WordPiece	Sub-word; 30,522-token vocab	Decomposes rare or misspelled words into sub-units, minimizing <unk>; bidirectional attention builds rich contextual embeddings suitable for classification.
GPT-2	Byte-level BPE	Sub-word; 50,257-token vocab	Operates on byte sequences, eliminating out-of-vocabulary tokens entirely and preserving generation fluency even for specialized words.

Model	Tokenizer	Strategy	Impact
Text-GAN	Custom word-level	Word-level; 1,504 tokens	Words outside the top 1,500 collapse to <unk>, pushing generated text toward generic, high-frequency words and limiting lexical diversity.

BERT's WordPiece and GPT-2's BPE both construct arbitrary words from sub-units, which is highly effective for movie reviews that frequently contain slang, typos, actor names, and creative punctuation (e.g., “acting-wise”, “oscar-worthy”). The Text-GAN's custom word-level vocabulary is far more constrained: a less common word such as “breathtaking” is seen only as <unk>, which measurably reduces output richness, as visible in the repetitive samples above.

7.2 Analysis of Metric Tradeoffs: Why GANs Struggle with Discrete Text

The Discrete Text Gradient Barrier

In standard GANs for images, the generator outputs continuous pixel values, so gradients flow cleanly from the discriminator's loss back to the generator's parameters. In text, the generator must instead output a categorical probability distribution over the vocabulary, and producing an actual token requires an argmax or multinomial sampling step. Because argmax is piecewise-constant, its gradient is zero everywhere, which breaks standard backpropagation between the discriminator and the generator's logits.

Soft-Embedding Approximation vs. Reinforcement Learning

To bypass this discrete barrier, this project's Text-GAN feeds the discriminator continuous soft-embedding vectors — weighted averages of word embeddings — rather than discrete tokens. While this restores differentiability, it introduces a major tradeoff: the discriminator quickly learns that real reviews have sharp, exact one-hot embeddings while fake reviews have fuzzy, distributed soft embeddings. Rather than learning grammar, it cheats by classifying sequences based on embedding variance, exactly the dynamic visible in the discriminator accuracy saturating near 1.0 almost immediately.

A more principled fix, used by SeqGAN, reformulates text generation as a reinforcement learning problem. The generator acts as an RL policy and the discriminator as a reward estimator; using the REINFORCE algorithm, the generator can be updated from reward feedback without requiring differentiable token outputs. The tradeoff is high reward variance and slower convergence.

Autoregressive GPT-2 (MLE Teacher Forcing) vs. Adversarial GANs

GPT-2 is trained via Maximum Likelihood Estimation (MLE) with teacher forcing. Since the target word is known at each step, the model simply maximizes its probability, and the resulting cross-entropy loss is smooth and provides strong, dense gradients at every token position. GANs instead search for a Nash equilibrium in a two-player game. In text, this game is inherently unstable because of the representation mismatch between the discrete target text and the generator's continuous approximations — a mismatch borne out empirically by the Text-GAN's near-zero BLEU/ROUGE scores compared to GPT-2's substantially higher scores.

7.3 Key Observations per Model

Model	Strengths	Limitations
BERT	Strong, high-precision classification from a pre-trained bidirectional encoder; captures nuanced sentiment with minimal fine-tuning.	Cannot generate text; recall lags precision after only one epoch; longer training time than the lightweight GAN.
GPT-2	Generates fluent, domain-relevant continuations after limited fine-tuning; pre-trained linguistic priors aid generation quality.	BLEU/ROUGE scores are inherently modest since these metrics favor extractive overlap with references rather than fluent generation.
Text-GAN	Demonstrates adversarial training for text with a differentiable soft-embedding workaround; extremely fast training (seconds per run).	Discrete token space makes training unstable; severe mode collapse observed; requires more sophisticated techniques (e.g., SeqGAN, reinforcement learning) to produce coherent text.

8. Conclusion

This experiment highlights the distinct strengths and weaknesses of three major NLP paradigms when applied to the same movie-review corpus:

- **BERT** excels at discriminative tasks such as sentiment classification, achieving 0.8621 precision and a 0.7772 F1-score after only a single fine-tuning epoch on 1,000 training samples. Its bidirectional attention mechanism and pre-trained representations make it the most practical choice when the goal is to classify or extract information from text.
- **GPT-2** demonstrates strong generative capabilities, producing fluent and contextually relevant review continuations. A perplexity of 81.16 after one epoch of fine-tuning shows meaningful, if early-stage, domain adaptation. The autoregressive architecture, trained via teacher forcing, is well-suited for tasks requiring text completion or creative writing within a specific domain.
- **Text-GAN** illustrates the fundamental challenges of applying adversarial training to discrete text sequences. While the architecture trained roughly 20× faster than BERT per run, the generator suffered from severe mode collapse, as evidenced by perfect discriminator accuracy and near-zero BLEU/ROUGE scores. This confirms the well-documented difficulty of text GANs: the non-differentiable nature of token sampling fundamentally disrupts the gradient signal that GANs rely upon, even with a soft-embedding workaround.

In summary, pre-trained transformer models like BERT and GPT-2 remain the preferred architectures for most NLP tasks due to their stable training dynamics and ability to leverage large-scale pre-training. GANs, while powerful in continuous domains such as image generation, require substantially more sophisticated techniques (such as reinforcement-learning-based training or SeqGAN) to produce coherent text.

9. References

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI Blog, 1(8), 9.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Nets. NeurIPS.
- Yu, L., Zhang, W., Wang, J., & Yu, Y. (2017). SeqGAN: Sequence Generative Adversarial Nets with Policy Gradient. AAAI.
- Kusner, M. J., & Hernández-Lobato, J. M. (2016). GANs for Sequences of Discrete Elements with the Gumbel-Softmax Distribution. arXiv:1611.04051.
- Sennrich, R., Haddow, B., & Birch, A. (2015). Neural Machine Translation of Rare Words with Subword Units. arXiv:1508.07909.
- Schuster, M., & Nakajima, K. (2012). Japanese and Korean Voice Search. ICASSP, 5149–5152.
- Pang, B., & Lee, L. (2005). Seeing Stars: Exploiting Class Relationships for Sentiment Categorization with Respect to Rating Scales. ACL, 115–124. (Rotten Tomatoes dataset, Hugging Face: cornell-movie-review-data/rotten_tomatoes)